UNIDAD 1
ACTIVIDAD PRÁCTICA DEL DIAGNÓSTICO NUMÉRICO AL CONTRATO DE PREPROCESADO

U-tad
CENTRO UNIVERSITARIO DE TECNOLOGÍA Y ARTE DIGITAL

Centro adscrito a
Universidad Camilo José Cela

ÍNDICE

ORGANIZACIÓN DE LA ACTIVIDAD PRÁCTICA 3

DESCRIPCIÓN DE LA ACTIVIDAD PRÁCTICA 3

EJERCICIO 1. INSTRUMENTACIÓN: MEDIR ANTES DE TOCAR 4

EJERCICIO 2. BARRIDO SISTEMÁTICO DE OPERACIONES PUNTUALES 5

EJERCICIO 3. DEL SENSOR A LA IMAGEN: MOSAICO DE BAYER Y DEMOSAICO 7

EJERCICIO 4. ROBUSTEZ DE UNA SEGMENTACIÓN POR COLOR EN HSV 8

EJERCICIO 5. EL CONTRATO DE PREPROCESADO 10

ENTREGABLES VERIFICABLES 11

RÚBRICA DE CORRECCIÓN 12

CÓMO REALIZAR LA ENTREGA 14

ORGANIZACIÓN DE LA ACTIVIDAD PRÁCTICA

Nombre de la práctica	Del diagnóstico numérico al contrato de preprocesado
Tipo de tarea	Individual
Entregables	Archivo único comprimido (.zip)
Peso en la evaluación	Según la guía docente de la asignatura

DESCRIPCIÓN DE LA ACTIVIDAD PRÁCTICA

En la Unidad 1 hemos recorrido el camino completo que va de los fotones a los números: la óptica, el sensor, el convertidor analógico-digital, el pipeline de la cámara y, ya en el terreno del software, las operaciones puntuales que reorganizan el histograma. Esta práctica pone ese recorrido a trabajar sobre un problema muy concreto.

El planteamiento es el siguiente. Nos entregan un pequeño lote de imágenes tomadas en condiciones desiguales y nos piden que definamos el preprocesado que se aplicará a todas ellas antes de cualquier análisis posterior. No basta con encontrar unos ajustes que dejen bonitas las fotos: hay que entregar una decisión defendible, con parámetros fijos, y demostrar con números que esa decisión mejora una tarea medible.

Ese matiz cambia la forma de trabajar. Cuando alguien dice "he aplicado gamma y la imagen ha mejorado", está describiendo una impresión. Cuando dice "he aplicado gamma con γ=0,55 y el porcentaje de píxeles pegados al valor 0 ha bajado del 18,4 % al 2,1 % mientras el rango ocupado entre los percentiles 1 y 99 pasaba de 96 a 213 niveles", está aportando una evidencia. La diferencia entre ambas frases es exactamente lo que esta práctica quiere ejercitar.

Sobre el enfoque de esta práctica. Cada ejercicio pide dos cosas: un resultado y la cifra que lo respalda. Las conclusiones que no vengan acompañadas de un número extraído de tu propio código no puntúan en el criterio de análisis, por muy bien redactadas que estén. A cambio, un resultado modesto, pero bien medido y honestamente interpretado puntúa igual que uno espectacular.

Material de partida

Necesitas cinco imágenes propias o de banco libre que cubran, entre todas, estas situaciones. Puede haber una imagen que cubra dos casos.

- Una imagen subexpuesta, con la mayor parte de la información apelotonada en la zona oscura del histograma.
- Una imagen sobreexpuesta o lavada, con poco contraste y el histograma comprimido.
- Una imagen con iluminación desigual: una parte bien expuesta y otra en sombra dentro del mismo encuadre.
- Una imagen con una dominante de color evidente (luz cálida de interior, luz de sombra azulada).
- Una imagen con un objeto de color saturado y bien definido sobre un fondo distinguible, que será la que uses en el Ejercicio 4.

Guarda los originales sin retocar en la carpeta de datos. Todo el procesamiento debe partir de ellos y ejecutarse por código.

[Figura sugerida: las cinco imágenes de partida con su histograma de luminancia al lado, en una rejilla 5×2]

EJERCICIO 1. INSTRUMENTACIÓN: MEDIR ANTES DE TOCAR

Antes de aplicar ninguna transformación necesitamos un instrumento de medida. Este ejercicio consiste en construirlo, y es la base de todos los demás: sin él, el resto de la práctica se convierte en una colección de impresiones.

La idea es sencilla. Cuando miramos un histograma, nuestro ojo detecta cuatro cosas: si hay píxeles amontonados contra los extremos, cuánto del rango disponible está realmente ocupado, si la distribución es ancha o estrecha y si hay uno o varios picos. Las cuatro se pueden cuantificar, y una vez cuantificadas se pueden comparar entre imágenes y entre transformaciones sin discutir sobre percepciones.

Qué tienes que implementar

Escribe una función `diagnostico(img)` que reciba una imagen y devuelva un diccionario con, al menos, estas métricas calculadas sobre el canal de luminancia:

Métrica	Qué mide	Cómo se calcula
sat_bajo	Porcentaje de píxeles pegados al negro, es decir, información perdida en sombras.	Fracción de píxeles con valor ≤ 2, expresada en tanto por ciento.
sat_alto	Porcentaje de píxeles quemados en blanco, es decir, información perdida en luces.	Fracción de píxeles con valor ≥ 253, expresada en tanto por ciento.
rango_p1_p99	Cuántos niveles del rango disponible ocupa realmente la imagen.	Diferencia entre el percentil 99 y el percentil 1 de las intensidades.
entropia	Cuánta información distinta contiene el histograma. Un histograma concentrado tiene entropía baja.	Entropía de Shannon del histograma normalizado de 256 niveles, en bits.
desv_tipica	Dispersión de intensidades, una medida directa de contraste global.	Desviación típica de los valores de luminancia.

Escribe también una función `diagnostico_color(img)` que devuelva la media de cada canal en RGB. La comparación entre esas tres medias es el indicio más directo de una dominante de color: si la media del canal azul está muy por debajo de las otras dos, la escena tira a cálido.

Cuidado con la luminancia. No uses la media aritmética de R, G y B como luminancia. Convierte a un espacio que separe luminancia y croma, como HSV (canal V) o YCrCb (canal Y), tal y como vimos en la unidad. Documenta cuál has elegido y por qué, porque esa elección condiciona todas las cifras que reportes después.

Qué tienes que entregar de este ejercicio

1. Una tabla con el diagnóstico de las cinco imágenes originales: una fila por imagen, una columna por métrica.
2. Un párrafo por imagen, de tres a cinco líneas, en el que traduzcas las cifras a un diagnóstico verbal. Por ejemplo: si la entropía es baja, el rango ocupado es estrecho y la desviación típica es pequeña, la imagen tiene poco contraste con independencia de si se ve clara u oscura.
3. Una predicción, para cada imagen, de qué operación esperas que funcione mejor y por qué. La comprobarás en el Ejercicio 2, y acertar o no acertar puntúa igual: lo que puntúa es que la predicción esté razonada.

EJERCICIO 2. BARRIDO SISTEMÁTICO DE OPERACIONES PUNTUALES

Con el instrumento construido, toca usarlo. En lugar de probar tres o cuatro ajustes a ojo y quedarnos con el que más nos guste, vamos a recorrer una rejilla completa de configuraciones, medir todas, y dejar que las cifras decidan.

Hay una razón práctica detrás de esto que va más allá del rigor académico. Cuando probamos ajustes de uno en uno, tendemos a parar en el primero que nos parece aceptable, y ese sesgo hace que rara vez encontremos el punto realmente bueno. Un barrido sistemático no tiene ese problema: explora también las zonas que nuestra intuición habría descartado.

La rejilla de configuraciones

Para cada una de las cinco imágenes, aplica todas estas configuraciones:

Operación	Parámetros a recorrer	Configuraciones
Corrección de gamma	$\gamma \in \{0{,}45; 0{,}6; 0{,}8; 1{,}0; 1{,}4; 1{,}8; 2{,}2\}$	7
Estiramiento lineal	percentiles de recorte $\in \{(0, 100), (1, 99), (2, 98), (5, 95)\}$	4
Ecualización global (HE)	sin parámetros	1
CLAHE	clipLimit $\in \{1{,}0; 2{,}0; 4{,}0\}$ × tileGrid $\in \{(4,4), (8,8), (16,16)\}$	9
Sin procesar (referencia)	la imagen original, para tener línea base	1

Son 22 configuraciones por imagen y 110 filas en total. Es precisamente por eso que el barrido tiene que hacerse con bucles y no a mano: si el código está bien escrito, añadir un valor de γ cuesta un carácter.

Todas las operaciones se deben hacer sobre luminancia. Aplica cada transformación sobre el canal de luminancia y reconstruye el color después. Ecualizar o estirar cada canal RGB por separado altera las relaciones entre ellos e introduce dominantes de color que no estaban en la escena. Si quieres comprobarlo, incluye un único ejemplo comparando ambas vías: es una de las figuras más elocuentes de esta práctica.

El fichero de resultados

Tu código debe generar un fichero `outputs/resultados_u1.csv` con una fila por combinación de imagen y configuración, y estas columnas exactas:

```
imagen,operacion,parametros,sat_bajo,sat_alto,rango_p1_p99,entropia,desv_tipica
IMG_01.jpg,original,-,18.42,0.31,96,5.12,24.8
IMG_01.jpg,gamma,g=0.45,1.05,0.44,213,6.94,52.3
IMG_01.jpg,clahe,clip=2.0;tile=8x8,0.87,0.52,241,7.21,58.1
...
```

El fichero debe salir del código. Un CSV escrito a mano se detecta con facilidad y anula el criterio de reproducibilidad.

El criterio de selección

Aquí está la parte que exige criterio propio. Tienes que definir, y escribir explícitamente en la memoria antes de mirar los resultados, qué entiendes por "mejor configuración". Una definición razonable podría ser: maximizar el rango ocupado sujeto a que ni sat_bajo ni sat_alto superen el 1 %. Otra podría priorizar la

entropía. Cualquiera es válida siempre que la declares por adelantado y la apliques de forma consistente a las cinco imágenes.

Después, aplica ese criterio a tu CSV mediante código, no a ojo, y presenta la configuración ganadora de cada imagen en una tabla resumen. Compárala con la predicción que hiciste en el Ejercicio 1 y comenta las discrepancias: los casos en los que te equivocaste suelen ser los más interesantes de analizar.

Protocolo de pies de figura. A partir de aquí, toda figura de la memoria debe llevar un pie que incluya: qué operación se aplicó, con qué parámetros exactos, y qué cambió en las cifras. Un pie que solo diga "resultado tras aplicar CLAHE" está incompleto. El pie correcto sería del tipo: "CLAHE con clipLimit=2,0 y tiles de 8×8 sobre el canal V. El rango entre percentiles pasa de 96 a 241 niveles y la saturación en negro cae del 18,4 % al 0,9 %."

EJERCICIO 3. DEL SENSOR A LA IMAGEN: MOSAICO DE BAYER Y DEMOSAICO

Este ejercicio baja un escalón en la cadena de formación de imagen. Hasta ahora hemos trabajado con imágenes RGB completas, como si cada píxel hubiera medido los tres canales. Sabemos por la unidad que no es así: en un sensor con matriz de filtros de color, cada fotosito mide un único canal y los otros dos se estiman por interpolación. Vamos a simular ese proceso y a medir cuánto cuesta.

La razón de hacerlo no es histórica. El demosaico es la primera fuente de error de la imagen con la que trabajamos, y sus artefactos se concentran justo donde después vamos a buscar bordes y estructuras. Entender dónde falla ayuda a interpretar resultados extraños en las unidades siguientes.

Paso 1: construir el mosaico

Parte de una de tus imágenes RGB y descarta información para simular lo que registraría el sensor. Con el patrón RGGB, la posición de cada píxel dentro del bloque 2×2 determina qué canal conserva:

```
import numpy as np

def simular_bayer(rgb):
    """Devuelve un mosaico 2D con un solo canal medido por posición.
    Patrón RGGB: R en (par, par), G en (par, impar) y (impar, par),
    B en (impar, impar). El resto de la información se descarta,
    igual que ocurre físicamente en el sensor."""
    alto, ancho = rgb.shape[:2]
    mosaico = np.zeros((alto, ancho), dtype=rgb.dtype)
    mosaico[0::2, 0::2] = rgb[0::2, 0::2, 0]   # R
    mosaico[0::2, 1::2] = rgb[0::2, 1::2, 1]   # G en filas pares
    mosaico[1::2, 0::2] = rgb[1::2, 0::2, 1]   # G en filas impares
    mosaico[1::2, 1::2] = rgb[1::2, 1::2, 2]   # B
    return mosaico
```

Antes de seguir, responde en la memoria a una pregunta: de los píxeles del mosaico, ¿qué fracción corresponde a cada canal? Comprueba tu respuesta contando sobre el array y relaciónala con lo que vimos sobre por qué el verde recibe más posiciones.

Paso 2: reconstruir por interpolación

Implementa un demosaico bilineal propio. Para cada píxel y cada canal que no midió, estima el valor promediando los vecinos que sí lo midieron. No uses la función de OpenCV para esta parte: el objetivo es que el mecanismo quede claro. Puedes usarla después, si quieres, como término de comparación.

Paso 3: medir el error

Ahora tenemos dos imágenes que deberían ser iguales pero no lo son: la original y la reconstruida. Cuantifica la diferencia:

1. Error absoluto medio por canal, entre la original y la reconstruida.
2. Un mapa de error: la diferencia absoluta agregada sobre los tres canales, mostrada como imagen en falso color.
3. Error absoluto medio calculado por separado en dos regiones: los píxeles que Canny o un simple umbral sobre la magnitud del gradiente marcarían como borde, y el resto. Compara ambas cifras.

El tercer punto es el interesante. La diferencia entre esas dos cifras es la que sostiene la afirmación de que el demosaico falla en los bordes, y es justamente el tipo de afirmación que hay que respaldar con un número en lugar de con una impresión visual.

EJERCICIO 4. ROBUSTEZ DE UNA SEGMENTACIÓN POR COLOR EN HSV

Segmentar un objeto por su color en HSV es sencillo mientras la iluminación no cambie. El problema aparece cuando el sistema se despliega y la luz de las cinco de la tarde no es la de las once de la mañana. Este ejercicio mide exactamente eso: cuánto aguantan unos rangos fijos ante un cambio de iluminación, y si un preprocesado adecuado los hace aguantar más.

Es el ejercicio que conecta toda la práctica. Las métricas del Ejercicio 1 y el barrido del Ejercicio 2 solo tienen sentido si desembocan en una tarea concreta que mejore de forma demostrable.

Paso 1: la máscara de referencia

Toma la imagen del objeto de color. Define los rangos HSV que aíslan el objeto: una ventana de tono, un mínimo de saturación para descartar los grises y una acotación de valor para manejar sombras y reflejos. Ajusta los rangos hasta que la máscara te convenza y anótalos. A partir de este momento quedan congelados: no puedes retocarlos en el resto del ejercicio.

Esa máscara es tu referencia. Guárdala en disco.

> ***Si tu objeto es rojo. El tono es una magnitud circular y el rojo cae justo en la costura, de modo que su rango envuelve el extremo de la escala. Necesitarás dos ventanas de tono y combinarlas con una operación lógica, en lugar de una sola. Documenta esa decisión: es una fuente de error muy habitual.***

Paso 2: perturbar la iluminación

Genera cinco versiones de la imagen aplicando gamma sobre la luminancia con $\gamma \in \{0{,}5; 0{,}7; 1{,}0; 1{,}4; 2{,}0\}$. La versión con $\gamma=1{,}0$ es la original y sirve de control. Estás simulando, de forma controlada y reproducible, que la escena se ha capturado con más o menos luz.

Paso 3: medir la degradación

Para cada versión perturbada, aplica exactamente los mismos rangos HSV que fijaste y calcula la intersección sobre unión (IoU) entre la máscara obtenida y la de referencia:

```
def iou(mascara_a, mascara_b):
    """IoU entre dos máscaras binarias. Vale 1 si coinciden por completo
    y 0 si no comparten ningún píxel. Nos sirve como medida objetiva
    de cuánto se ha degradado la segmentación."""
    a = mascara_a.astype(bool)
    b = mascara_b.astype(bool)
    interseccion = np.logical_and(a, b).sum()
    union = np.logical_or(a, b).sum()
    return interseccion / union if union > 0 else 0.0
```

Representa la IoU frente a γ. Tendrás una curva con el máximo en $\gamma=1{,}0$ que cae hacia los extremos. La forma de esa caída, y lo rápido que se produce, es el resultado principal del ejercicio.

Paso 4: la intervención

Ahora repite todo el proceso con una diferencia: antes de convertir a HSV y aplicar los rangos, normaliza la luminancia con la mejor configuración que encontraste en el Ejercicio 2 para esta imagen. Vuelve a calcular la IoU para los mismos cinco valores de γ.

Dibuja ambas curvas en la misma gráfica. Y entonces responde, con las cifras delante, a la pregunta que da sentido a toda la práctica: ¿ha servido de algo el preprocesado? Cuantifica la respuesta. Por ejemplo, la IoU media a lo largo de los cinco valores de γ antes y después, o la IoU en el caso más desfavorable.

> ***Un resultado negativo bien argumentado vale lo mismo. Es perfectamente posible que en tu imagen el preprocesado no mejore la robustez, o que la mejore solo en un extremo del rango. Si es así, dilo y explica por qué crees que ocurre. Lo que no puntúa es afirmar una mejora que las cifras no respaldan.***

EJERCICIO 5. EL CONTRATO DE PREPROCESADO

El último ejercicio es el más corto de programar y el más exigente de escribir. Consiste en cerrar la práctica con una decisión.

Escribe una única función `preprocesar(img)` que aplique el preprocesado que has decidido adoptar para todo el lote. Sin parámetros libres: los valores concretos van dentro, fijados. Es la función que entregarías a quien tenga que construir la etapa siguiente del sistema.

Acompáñala de un documento breve, `memoria/contrato_preprocesado.md`, de una página como máximo, con esta estructura:

1. Qué hace la función, en dos o tres frases y sin jerga.
2. Qué parámetros lleva fijados y de qué fila del CSV de resultados sale cada uno.
3. Qué mejora aporta, con las cifras del Ejercicio 4: IoU antes y después.
4. En qué condiciones deja de funcionar. Este punto es obligatorio y no admite un "en ninguna". Toda decisión de preprocesado tiene un dominio de validez, y reconocerlo es parte del trabajo.
5. Qué medirías para detectar que ha dejado de funcionar una vez desplegado. Basta con nombrar una o dos de las métricas del Ejercicio 1 y el umbral que encendería la alarma.

ENTREGABLES VERIFICABLES

Además de la memoria, la entrega debe contener estos artefactos. Se comprueban de forma directa y su ausencia se penaliza en el criterio correspondiente.

Artefacto	Qué debe contener	Criterio afectado
outputs/resultados_u1.csv	Las 110 filas del barrido del Ejercicio 2 con las columnas especificadas, generadas por código.	Evidencia cuantitativa
outputs/robustez_hsv.csv	IoU para cada valor de γ, sin y con normalización previa. Diez filas.	Evidencia cuantitativa
memoria/contrato_preprocesado.md	El documento de una página del Ejercicio 5, con los cinco apartados.	Análisis
codigo/requirements.txt	Versiones fijadas de las bibliotecas utilizadas. Sin instalaciones dentro de las celdas.	Reproducibilidad
Semilla fijada	Una semilla declarada al principio del código, aunque en esta práctica haya poca aleatoriedad. Es un hábito que se consolida ahora.	Reproducibilidad
Figuras con pie completo	Toda figura con operación, parámetros exactos y cifra del cambio observado.	Comunicación

Lista de comprobación antes de entregar

- Reinicia el kernel y ejecuta el notebook de principio a fin. Si falla en algún punto, la entrega no es reproducible.
- Comprueba que las figuras del directorio de salida se han regenerado en esa ejecución y no son restos de una anterior.
- Verifica que todas las cifras que citas en la memoria aparecen en alguno de los CSV entregados.
- Revisa que ningún pie de figura se limita a nombrar la operación sin dar parámetros ni cifras.
- Confirma que la memoria va en PDF y que el zip sigue el nombre indicado.

RÚBRICA DE CORRECCIÓN

La rúbrica sigue los criterios listados a continuación, con distintos pesos sobre la nota final.

Criterios	Excelente	Satisfactorio	No satisfactorio	Insuficiente
Corrección técnica y completitud (3 puntos)	De 2,5 a 3 puntos. Las cinco métricas de diagnóstico implementadas y correctas. Barrido completo de las 22 configuraciones sobre las cinco imágenes. Mosaico de Bayer y demosaico bilineal propios y funcionando. Segmentación HSV con las dos ventanas de tono si el objeto lo requiere. Todo opera sobre luminancia.	De 2 a 2,5 puntos. Flujo correcto con fallos menores: alguna métrica mal normalizada, alguna configuración del barrido ausente, o demosaico resuelto con la función de biblioteca en lugar de implementado.	De 1,5 a 2 puntos. Omisiones relevantes: barrido parcial, realce aplicado por canal RGB sin justificarlo, o ausencia de uno de los cinco ejercicios.	De 0 a 1,5 puntos. Faltan bloques centrales o el código no ejecuta de principio a fin.
Evidencia cuantitativa y análisis (4 puntos)	De 3,5 a 4 puntos. Toda afirmación va respaldada por una cifra propia. El criterio de selección está declarado antes de mirar los resultados y se aplica por código. Las curvas de IoU se interpretan cuantificando la mejora o su ausencia. Las discrepancias con la predicción inicial se analizan.	De 2,5 a 3,5 puntos. Análisis correcto con lagunas: se dan cifras pero no siempre se comparan, o el criterio de selección se aplica a ojo sobre la tabla.	De 2 a 2,5 puntos. Comentarios de tipo mejor o peor sin cifras que los sostengan. El CSV existe pero la memoria no lo utiliza.	De 0 a 2 puntos. Sin análisis, o conclusiones que las propias cifras entregadas contradicen.
Calidad de figuras y comunicación (1,5 puntos)	De 1,25 a 1,5 puntos. Figuras legibles, con escalas coherentes entre paneles comparables, y pies que incluyen operación, parámetros y cifra del cambio.	De 1 a 1,25 puntos. Presentación clara con alguna incoherencia menor en escalas o algún pie incompleto.	De 0,75 a 1 puntos. Figuras sin rotular, escalas inconsistentes o pies que solo nombran la operación.	De 0 a 0,75 puntos. Resultados difíciles de interpretar o figuras ausentes.

Criterios	Excelente	Satisfactorio	No satisfactorio	Insuficiente
	Histogramas con ejes rotulados.			
Reproducibilidad y estilo de código (1,5 puntos)	De 1,25 a 1,5 puntos. Código modular con funciones reutilizables, rutas relativas, semilla fijada, requirements con versiones y ejecución verificada de principio a fin.	De 1 a 1,25 puntos. Código limpio con pequeños atajos, o requirements sin versiones fijadas.	De 0,5 a 1 puntos. Ejecuta pero es monolítico, con rutas absolutas o dependencias instaladas en celdas.	De 0 a 0,5 puntos. No reproducible: falla al ejecutar en limpio o los resultados no se pueden regenerar.

CÓMO REALIZAR LA ENTREGA

Archivo único comprimido (.zip) con nombre: **U1_Practica_Apellidos_Nombre.zip**

Estructura interna obligatoria:

```
U1_Practica_Apellidos_Nombre/
├─ memoria/
│   ├─ informe.pdf
│   └─ contrato_preprocesado.md
├─ codigo/
│   ├─ auditoria_fotometrica.ipynb   (o .py)
│   └─ requirements.txt
├─ data/
│   └─ (las cinco imágenes originales sin retocar)
└─ outputs/
    ├─ resultados_u1.csv
    ├─ robustez_hsv.csv
    └─ (figuras generadas: barrido_img01.png, error_demosaico.png, ...)
```

Memoria: PDF (no Word). Es el formato exigido; una entrega en otro formato se penaliza en el criterio de comunicación.

Código: Jupyter Notebook (.ipynb) o script (.py) ejecutable de principio a fin, sin instalar dependencias dentro de las celdas.

Figuras: PNG o JPG con nombres autoexplicativos, generadas por el propio código.

Resultados: los ficheros CSV que se piden en cada ejercicio deben venir generados por el código, no escritos a mano.

Límites prácticos: tamaño total ≤ 200 MB. No incluyas pesos de modelos salvo que se pidan explícitamente.